

✓ Lab: Extreme Events

Modeling the Unusual

- For a given sample, the maximum and minimum are fixed
- But if we have a large number of samples, we can study the distribution/density of those max/min draws
- Imagine:
 - The biggest storm in a year
 - The worst loss for the market in a month
 - The biggest earthquake in a decade
 - The longest time between goals in a game
- This is how we understand the distribution/density of extreme events
- The **k -th order statistic** is the k -th largest value in a sample of n from some distribution
- The minimum is the first order statistic and the maximum is the n -th order statistic

✓ Example: Drawing from a Gaussian

Below, we find the minimum of a sample of size n drawn from a normal distribution, and then repeat this b times, to get a distribution of the minimum with 10000 counts in it. This is the distribution of the 1st order statistic, shown below in blue.

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

rng = np.random.default_rng( seed = 100 ) # Create a random number stream

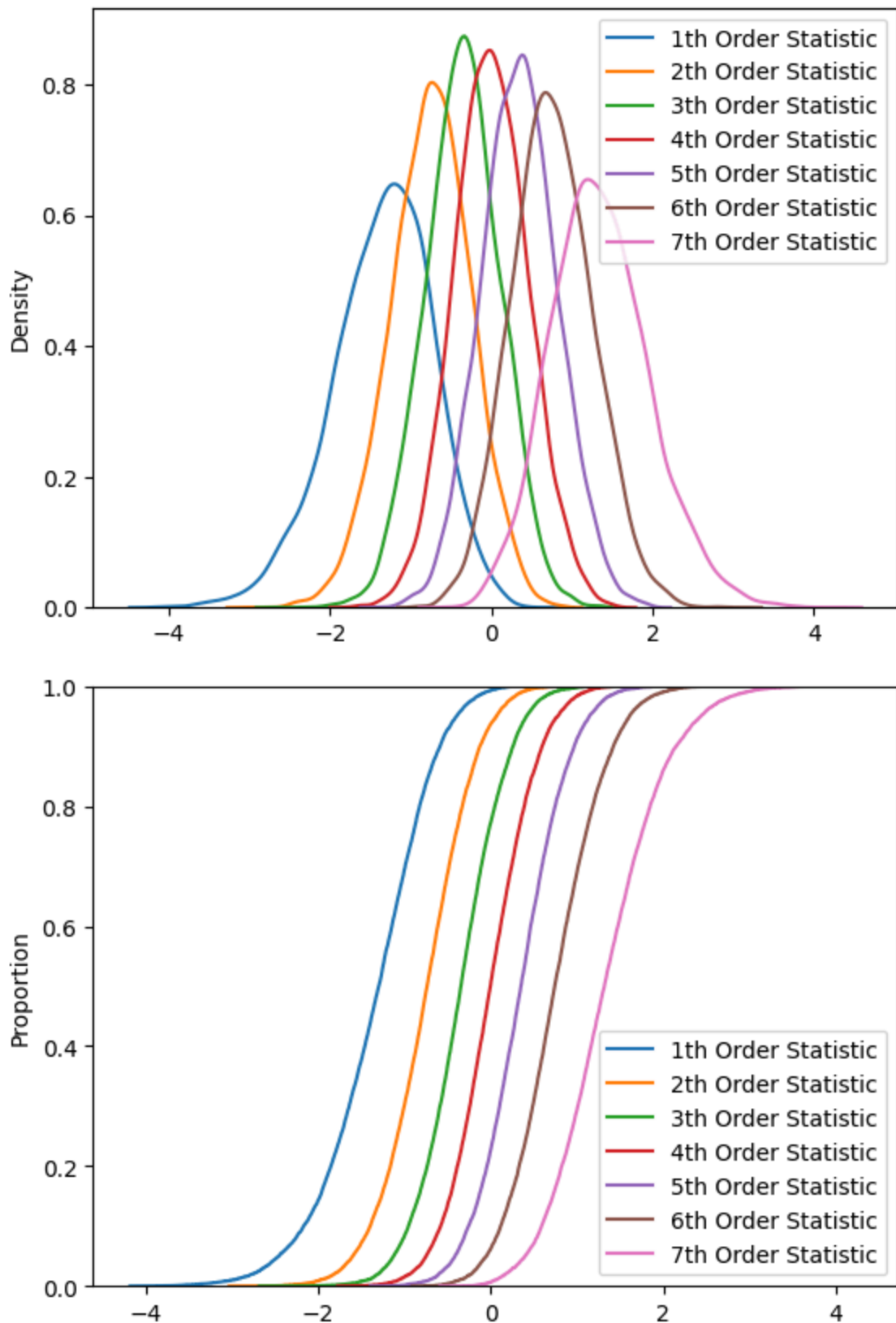
n = 7 # Sample Size
b = 10_000 # Number of Simulations

sample = rng.standard_normal( size = (n,b) ) # Draw a sample from the standard
sorted_sample = np.sort(sample, axis=0) # Sort each column (each simulated samp

for j in range(n): # For each rank
```

```
sns.kdeplot( sorted_sample[j,:], label=f'{1+j}th Order Statistic') # Plot t
plt.legend() # Add legend
plt.show()

for j in range(n): # For each rank
    sns.ecdfplot( sorted_sample[j,:], label=f'{1+j}th Order Statistic') # Plot
plt.legend() # Add legend
plt.show()
```



Analytic Relationship

Maximum

If $F(x)$ is the underlying true CDF, let's consider the analytical equation for the distribution of the maximum of a sample of size n . (The maximum is its own random variable, with its distribution plotted above. What is this in math?) To build the CDF of the maximum, we ask (as always): "What's the probability the maximum lands at or below x ?" for every x . In other words:

$$F_{max}(x) = P(X_{max} \leq x)$$

This is nothing new, just applying it to the distribution of a specific order statistic, the max.

So what is the **probability that all of the draws are less than or equal to x** , i.e. what is the probability that the maximum is less than or equal to x (the right hand side)?

Because each draw is independent:

$$P(X_1 \leq x, \dots, X_n \leq x) = P(X_1 \leq x) \cdot P(X_2 \leq x) \cdot \dots \cdot P(X_n \leq x)$$

Because we are pulling from the same distribution every time, **each of these probabilities equals our CDF, $F(x)$** :

$$P(X_1 \leq x, \dots, X_n \leq x) = F(x) \cdot F(x) \cdot \dots \cdot F(x)$$

The probability that all of the draws are less than or equal to x is therefore:

$$F_{max}(x) = F(x) \cdot F(x) \cdot \dots \cdot F(x) = F(x)^n$$

What is its density?

Minimum

The distribution of the minimum is somewhat harder, because our CDF is asking $\leq x$, and " $\max \leq x$ " means "all $\leq x$ " but " $\min \leq x$ " does not mean "all $\leq x$ ". We can't just substitute in multiple copies of $F(x)$. Instead of looking at the probability that all draws are less than or equal to x , we need to know the probability that at least one is.

We want to ask "is the minimum less than or equal to x ?" which is the opposite of asking "are all of the draws, including the minimum, above x ?" The probability that all observations are above x is $(1 - F(x))^n$. Because this is the opposite of the question we actually want to ask, the complementary probability is what we truly want:

$$F_{min}(x) = 1 - (1 - F(x))^n$$

What is its density?

All of the other order statistics, generally

- If you're curious, the density of the k -th of n order statistics is

$$f_k(x) = \frac{n!}{(k-1)!(n-k)!} F(x)^{k-1} (1 - F(x))^{n-k} f(x)$$

- We'll use the results for the minimum and maximum

✓ The Data

We have three datasets:

1. Rainfall *totals* for every state-month from 1981 to 2025, with 5 reporting stations per state each month
2. Rainfall for every *station-day* over that same 1981-2025 period (same 5 stations per state) — much bigger, and used for a single question near the end of Step One
3. Daily and monthly U.S. stock market returns from 1926 to 2026

We want to understand tail behavior, to better reason about risk (engineering, financial, etc.).

```
df = pd.read_parquet("ghcn_prcp_monthly.parquet")
len(df)
df.head(20)
```

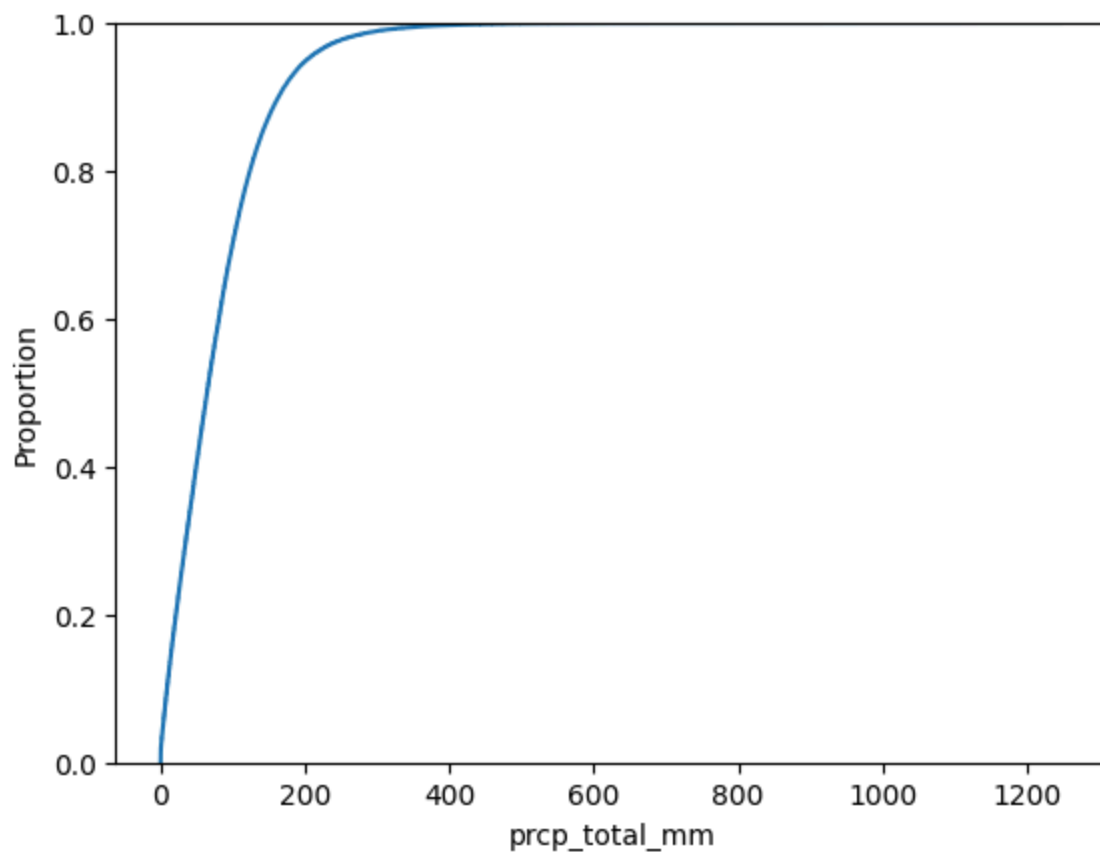
	state	state_name	station_rank	station_id	station_name	latitude	longi
0	AK	Alaska	1	USW00027502	BARROW AP	71.2869	-156
1	AK	Alaska	1	USW00027502	BARROW AP	71.2869	-156
2	AK	Alaska	1	USW00027502	BARROW AP	71.2869	-156
3	AK	Alaska	1	USW00027502	BARROW AP	71.2869	-156
4	AK	Alaska	1	USW00027502	BARROW AP	71.2869	-156
5	AK	Alaska	1	USW00027502	BARROW AP	71.2869	-156
6	AK	Alaska	1	USW00027502	BARROW AP	71.2869	-156
7	AK	Alaska	1	USW00027502	BARROW AP	71.2869	-156
8	AK	Alaska	1	USW00027502	BARROW AP	71.2869	-156
9	AK	Alaska	1	USW00027502	BARROW AP	71.2869	-156
10	AK	Alaska	1	USW00027502	BARROW AP	71.2869	-156
11	AK	Alaska	1	USW00027502	BARROW AP	71.2869	-156
12	AK	Alaska	2	USW00027401	BARTER ISLAND WSO AP	70.1333	-143
13	AK	Alaska	2	USW00027401	BARTER ISLAND WSO AP	70.1333	-143
14	AK	Alaska	2	USW00027401	BARTER ISLAND WSO AP	70.1333	-143
15	AK	Alaska	2	USW00027401	BARTER ISLAND WSO AP	70.1333	-143
16	AK	Alaska	2	USW00027401	BARTER ISLAND WSO AP	70.1333	-143
17	AK	Alaska	2	USW00027401	BARTER ISLAND WSO AP	70.1333	-143
18	AK	Alaska	2	USW00027401	BARTER ISLAND WSO AP	70.1333	-143
19	AK	Alaska	2	USW00027401	BARTER ISLAND WSO AP	70.1333	-143

Step One.

1. Plot the ECDF and KDE of `prcp_total_mm` with seaborn— the distribution and density of rainfall across the entire U.S.

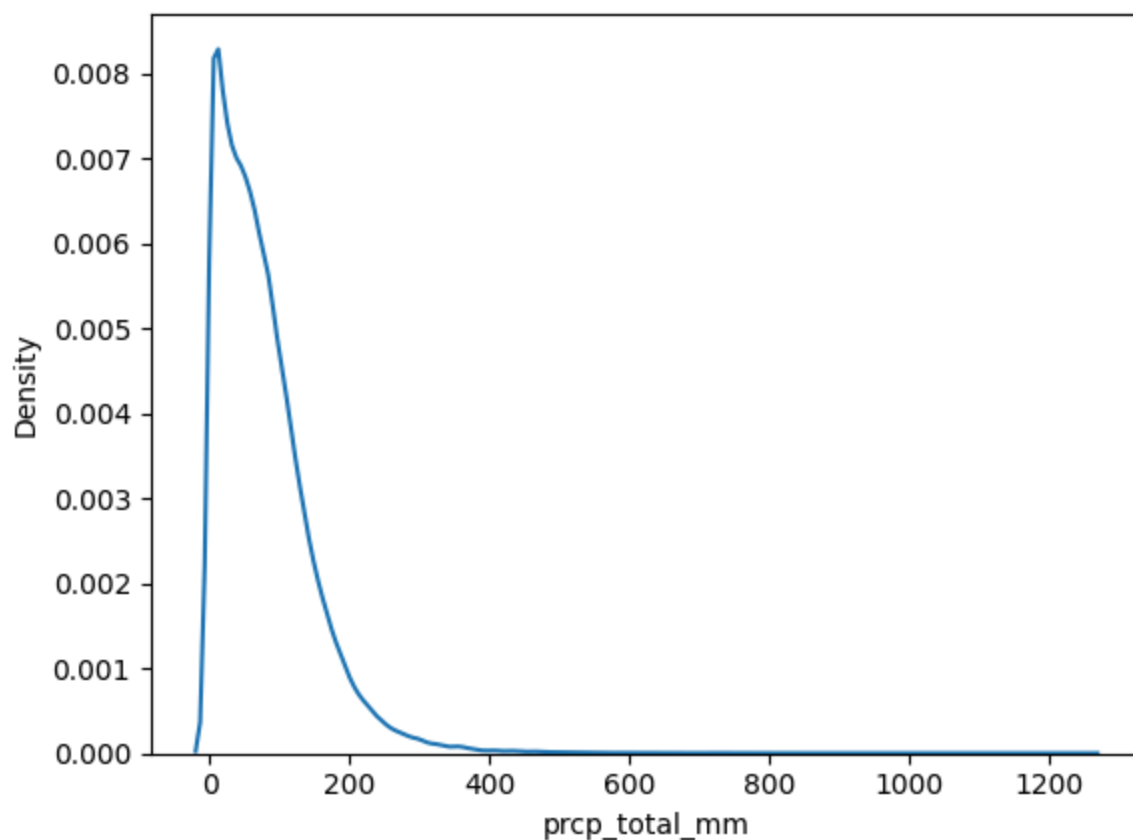
```
sns.ecdfplot(data=df, x="prcp_total_mm")
```

```
<Axes: xlabel='prcp_total_mm', ylabel='Proportion'>
```



```
sns.kdeplot(data=df, x="prcp_total_mm")
```

<Axes: xlabel='prcp_total_mm', ylabel='Density'>

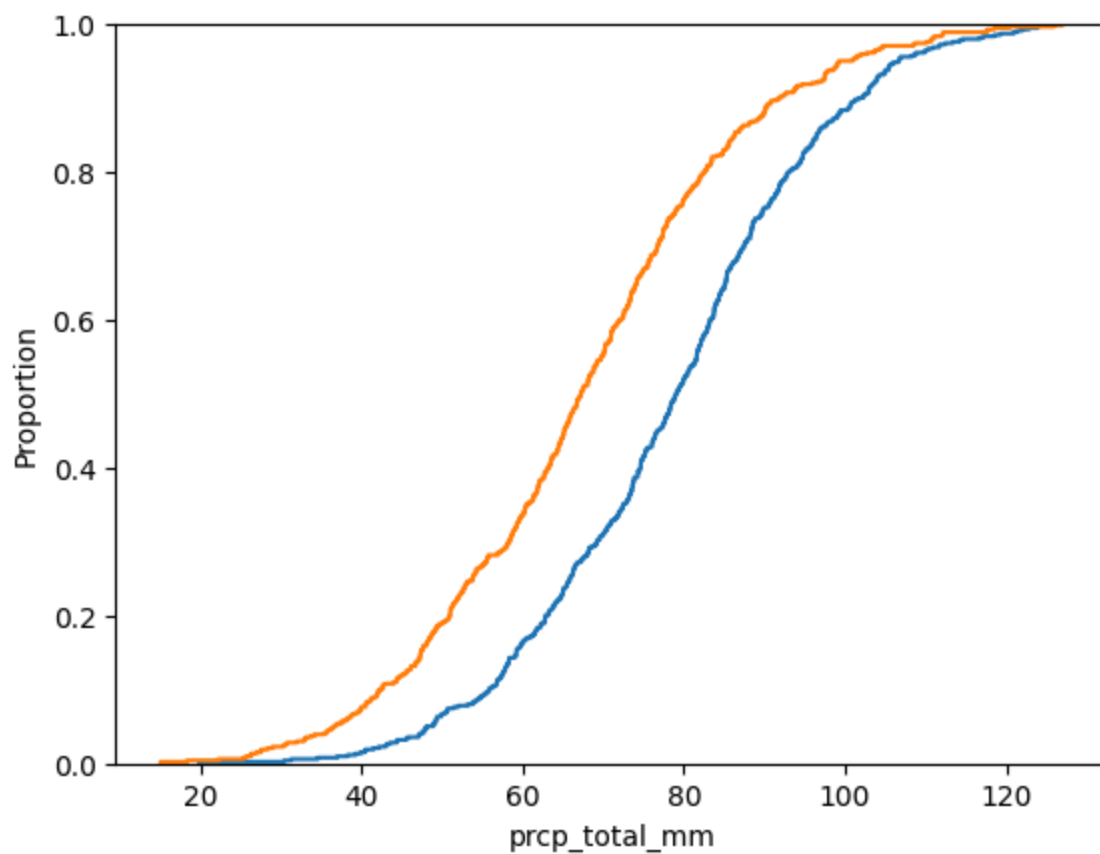


2. Now group by year-month and compute the **median** and **maximum** precipitation nationwide. Plot the ECDF and KDE of these two statistics.

```
mean_df = df.groupby(["year", "month"])["prcp_total_mm"].mean()
med_df = df.groupby(["year", "month"])["prcp_total_mm"].median()
```

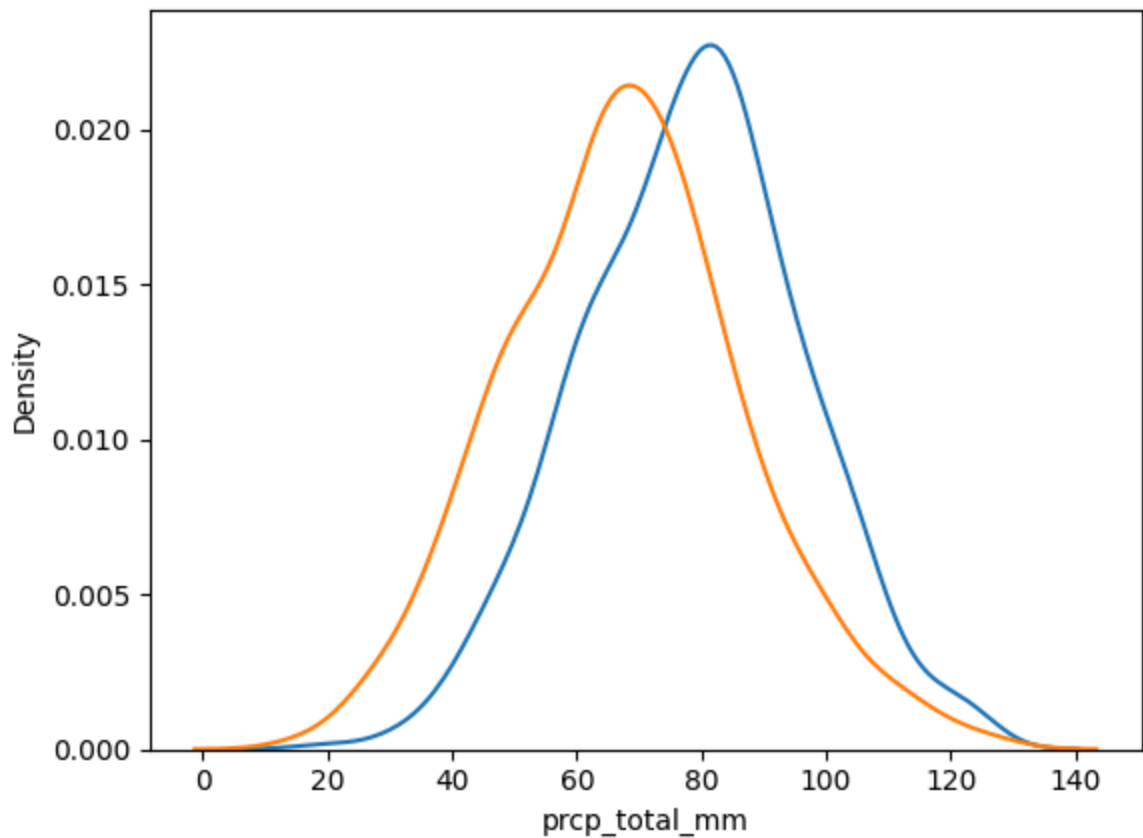
```
sns.ecdfplot(mean_df)
sns.ecdfplot(med_df)
```

<Axes: xlabel='prcp_total_mm', ylabel='Proportion'>



```
sns.kdeplot(mean_df)  
sns.kdeplot(med_df)
```

```
<Axes: xlabel='prcp_total_mm', ylabel='Density'>
```



3. How do your results in part 2 compare to your results in part 1?

Compare and contrast the ECDFs and KDEs from parts 1 and 2. Explain why they are different:

Answer:

Compared to the axes from part 1, part 2's axes are on a much smaller scale of total precipitation, ranging from 0-120/140 instead of 0-1200. The reason for this is that part 2's ECDFs and KDEs use the means and medians of the precipitation instead of the raw values. The ECDFs and KDEs also look different. The ECDF in part 1 is extremely skewed compared to part 2's ECDFs. The KDEs have a peak that is more centered and isn't too far on one side, and overall look like a normal distribution. The difference in the ECDFs is explained by the majority of observed values being small in the original ECDF. The differences in KDEs can be explained by the mean and median making the range of values narrower and excluding extreme values, and also because averaging values goes towards a normal distribution.

4. Now define a state's rainfall differently: take the **median** of the 5 stations in each month, then find the **maximum** across the state's 12 months in a year — the "wettest month." For each state, what's the expected value of the wettest month?

Plot the ECDF of Virginia's annual maxima (the wettest month in each of the 45 years), and overlay the analytical prediction $F_{\max}(x) = \hat{F}(x)^{12}$.

```
# One rainfall value per state-month, taking the median of the stations
state_month = df.groupby(["state", "year", "month"], as_index=False)["prcp_total_

# Maximum of the 12 state-month observations above, for each year: the wettest
state_year_max = state_month.groupby(["state", "year"], as_index=False).max() #

exp_wettest = state_year_max.groupby("state", as_index=False).agg(E_wettest_mor
exp_wettest.loc[exp_wettest["state"] == "VA"]

state = "VA"

levels = (
    state_month.loc[state_month["state"] == state, "prcp_total_mm"]
    .to_numpy()
)

observed_max = (
    state_year_max.loc[state_year_max["state"] == state, "prcp_total_mm"]
    .to_numpy()
)

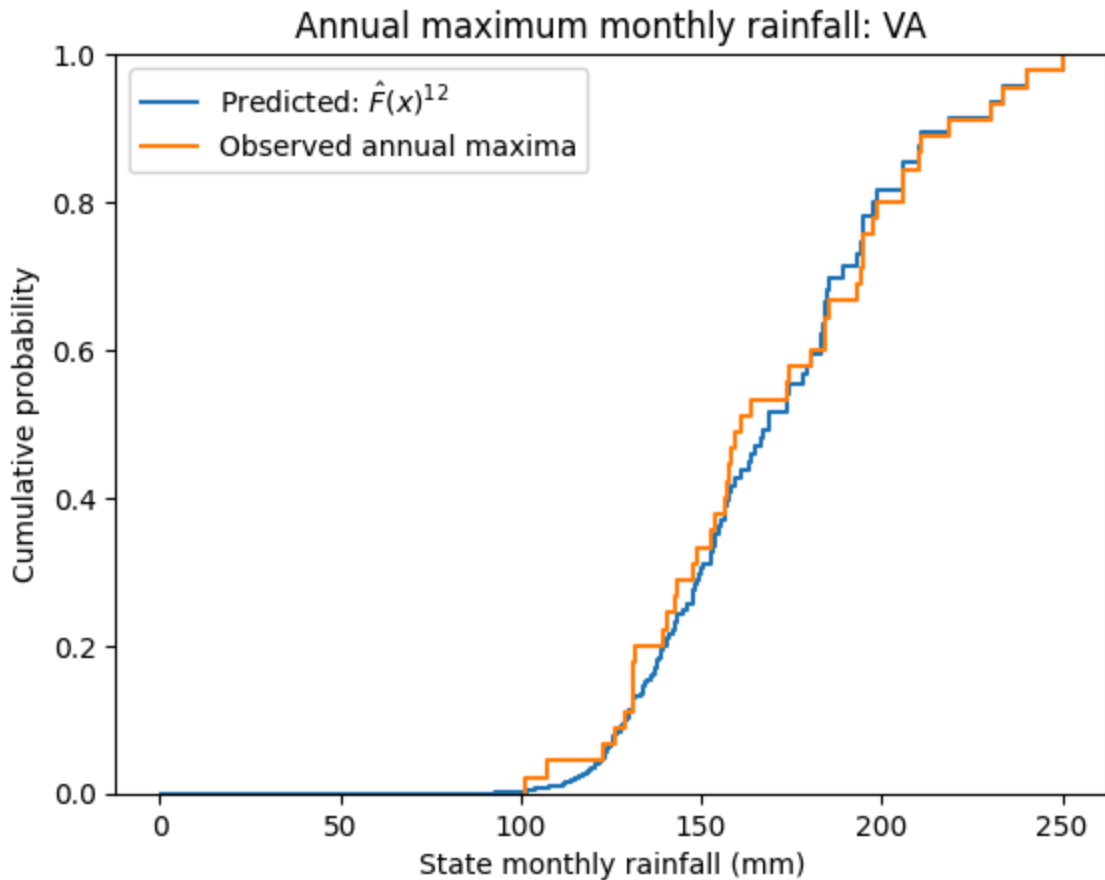
x = np.sort(np.unique(levels))
F_hat = np.searchsorted(np.sort(levels), x, side="right") / len(levels)

plt.step(
    x,
    F_hat**12,
    where="post",
    label=r"Predicted: $\hat{F}(x)^{12}$"
)

sns.ecdfplot(x=observed_max,
    label="Observed annual maxima")

plt.xlabel("State monthly rainfall (mm)")
plt.ylabel("Cumulative probability")
plt.title(f"Annual maximum monthly rainfall: {state}")
plt.legend()
plt.show()

print(state_year_max)
```



	state	year	month	prcp_total_mm
0	AK	1981	12	45.099998
1	AK	1982	12	44.599998
2	AK	1983	12	39.000000
3	AK	1984	12	72.500000
4	AK	1985	12	55.599995
...	...	...	...	...
2245	WY	2021	12	31.400000
2246	WY	2022	12	45.099998
2247	WY	2023	12	103.500008
2248	WY	2024	12	28.500002
2249	WY	2025	12	42.099998

[2250 rows x 4 columns]

How close is the fit here? If it differs from question 4's fit, why:

Answer

The fit is very close, with the predictions just slightly underpredicting for smaller maxima and slightly overpredicting larger maxima. For the most part, they overlap or are close to overlapping.

5. **A question for FEMA.** So far you've worked with *monthly* rainfall totals. Switch now to the daily station-level file (`ghcn_prcp_daily.parquet`) — reload `df` from it,

and don't mix it with the monthly data above.

A day with more than 120mm of rain is a serious flood/landslide risk. On any single station-day, that's rare. But a state doesn't get one chance a month — it gets 5 stations $\times$ (days in the month) chances. FEMA doesn't care about the average day, or the single worst day on record; it cares about the probability that *some* station has a disaster day at some point during the month.

For each state-month, estimate

$$P(\text{at least one station-day exceeds 120mm}) = 1 - \hat{F}(120)^n$$

where $\hat{F}$ is the empirical CDF of a single station-day's rainfall for that state and calendar month (pooled across all years), and n is the number of station-days in one occurrence of that month (5 stations $\times$ days in the month). Report the 20 highest-risk state-months.

```
# Fresh load -- this question uses only the daily file, not the monthly one abc
#df_daily = pd.read_parquet("data/ghcn_prcp_daily.parquet")

url = (
    "https://www.dropbox.com/scl/fi/"
    "pjmyop3fqjdy8oabkc4ab/ghcn_prcp_daily.parquet"
    "?rlkey=otxenczb3pxgzz53kc1i5zxww&dl=1"
)
df_daily = pd.read_parquet(url)
df_daily = df_daily.loc[df_daily["observed"]]

threshold = 120 # mm in a single day

def disaster_risk(group):
    F_hat = (group["prcp_mm"] <= threshold).mean()
    n = len(group) / group["year"].nunique() # station-days in one year's occurrence
    return 1 - F_hat**n

risk = (
    df_daily.groupby(["state", "month"])
        .apply(disaster_risk, include_groups=False)
        .rename("p_any_station_day_exceeds_120mm")
        .reset_index()
        .sort_values("p_any_station_day_exceeds_120mm", ascending=False)
)

sns.kdeplot(risk['p_any_station_day_exceeds_120mm'])

risk.head(20)
```


Which states and months face the most risk of a greater-than-120mm day? How high is the probability? Among all state-months, describe the density of high-precipitation days:

Answer

state	month	p any station day exceeds 120mm
FL	9	0.373367

Hawaii generally faces the largest risk of getting more than 120 mm of rain in a day across several different months. Florida and Louisiana regularly face large risks. The risks appear to be highest around the spring and fall months 3-5 and 9-11, and risks are lowest during the winter months 12-2, with the exception of Hawaii who of course gets more rain instead of snow. These high-risk states and months have probabilities from 0.3 - 0.4. According to the KDE, the probability of a high-precipitation day is around 0 - 0.5.

Step Two.

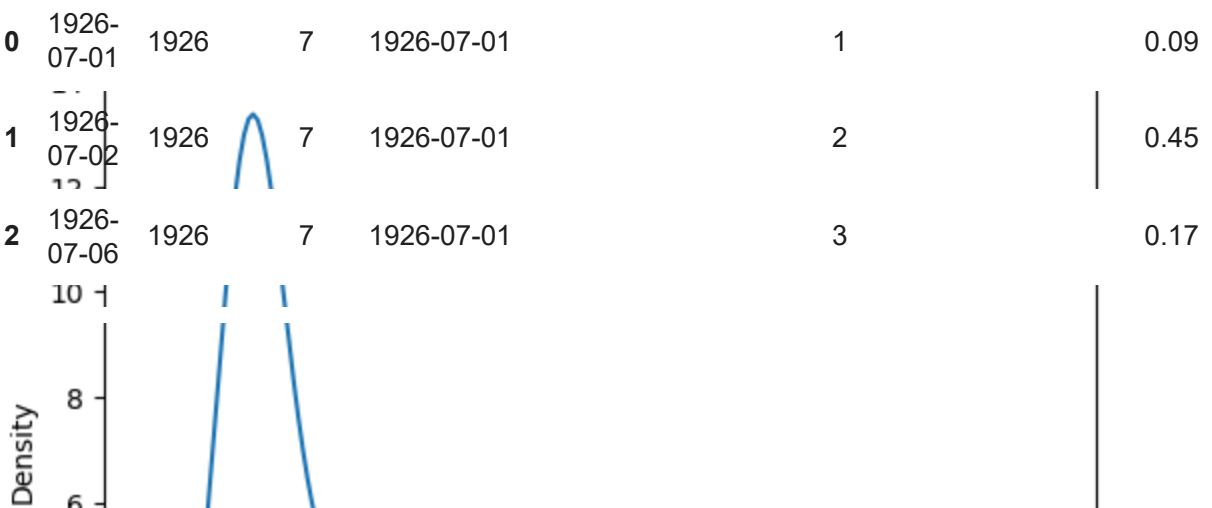
state	month	p any station day exceeds 120mm
LA	4	0.299512

Our second data set is daily and monthly U.S. stock market returns. Instead of storms across space (five stations in a state) or across the calendar (twelve months in a year), the "sample" is the set of trading days within a month, and we ask about the best and worst day in that sample.

```
df = pd.read_parquet("ff_market_daily.parquet")

print(f"{len(df):,} trading days")
print(f"{df['date'].min().date()} through {df['date'].max().date()}")
df.head()
```

date	year	month	month_start	trading_day_of_month	mkt_excess_return_pct
1926-07-01	1926	7	1926-07-01	1	0.09
1926-07-02	1926	7	1926-07-01	2	0.45
1926-07-06	1926	7	1926-07-01	3	0.17



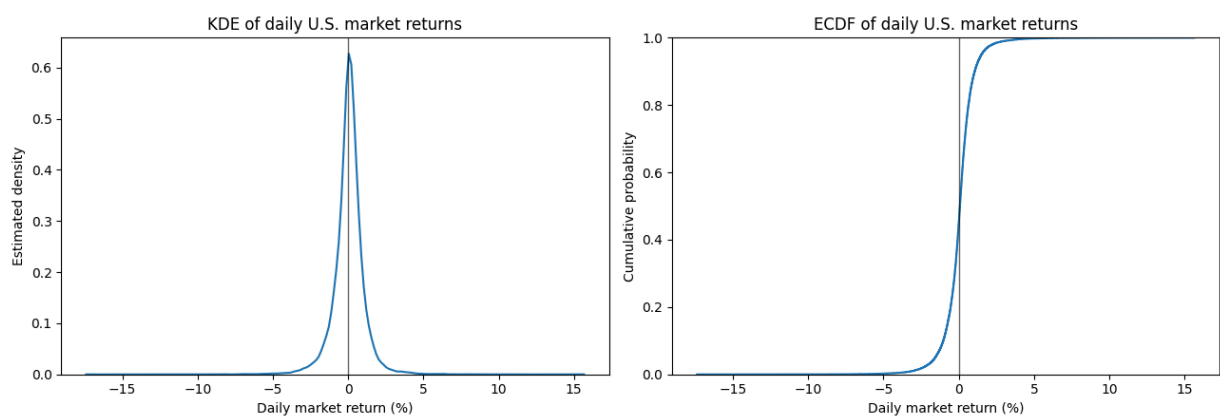
1. Plot the KDE and ECDF of daily market returns.

```
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

sns.kdeplot(
    data=df,
    x="market_return_pct",
    cut=0,
    ax=axes[0],
)
axes[0].axvline(0, color="black", linewidth=1, alpha=0.6)
axes[0].set(
    title="KDE of daily U.S. market returns",
    xlabel="Daily market return (%)",
    ylabel="Estimated density",
)

sns.ecdfplot(
    data=df,
    x="market_return_pct",
    ax=axes[1],
)
axes[1].axvline(0, color="black", linewidth=1, alpha=0.6)
axes[1].set(
    title="ECDF of daily U.S. market returns",
    xlabel="Daily market return (%)",
    ylabel="Cumulative probability",
)

plt.tight_layout()
plt.show()
```



2. For each calendar month, compute the **best** (max) and **worst** (min) daily return — the first and n -th order statistics of that month's trading days. Plot the KDE of the monthly best and worst returns, overlaid.

```
monthly = (
    df.groupby(["year", "month"], as_index=False)
        .agg(
            month_start=("month_start", "first"),
            trading_days=("date", "size"),
            worst_return_pct=("market_return_pct", "min"),
            best_return_pct=("market_return_pct", "max"),
            median_return_pct=("market_return_pct", "median"),
        )
)

# Put losses and gains on the same positive-magnitude scale.
monthly["worst_loss_pct"] = -monthly["worst_return_pct"]

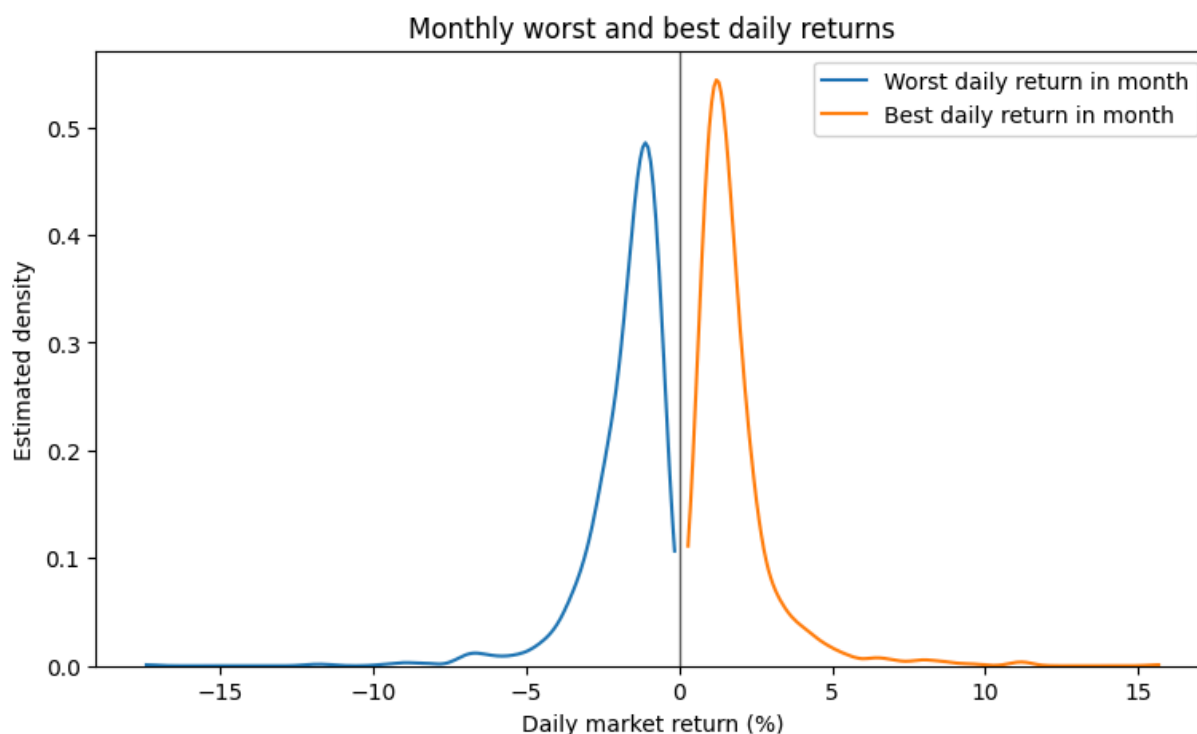
print(f"{len(monthly):,} calendar months")
print(
    f"Trading days per month: {monthly['trading_days'].min()} "
    f"to {monthly['trading_days'].max()}"
)
monthly.head()
```

1,201 calendar months
Trading days per month: 15 to 27

	year	month	month_start	trading_days	worst_return_pct	best_return_pct	me
0	1926	7	1926-07-01	25	-0.72	1.10	
1	1926	8	1926-08-01	26	-1.39	0.85	
2	1926	9	1926-09-01	24	-0.98	0.91	
3	1926	10	1926-10-01	25	-1.81	1.53	
4	1926	11	1926-11-01	24	-0.62	0.67	

```
plt.figure(figsize=(9, 5))
sns.kdeplot(
    x=monthly["worst_return_pct"],
    label="Worst daily return in month",
    cut=0,
)
sns.kdeplot(
    x=monthly["best_return_pct"],
    label="Best daily return in month",
    cut=0,
)
```

```
plt.axvline(0, color="black", linewidth=1, alpha=0.6)
plt.xlabel("Daily market return (%)")
plt.ylabel("Estimated density")
plt.title("Monthly worst and best daily returns")
plt.legend()
plt.show()
```



Describe the symmetry (or lack of it) between the best- and worst-day densities. Compare them to the overall populations of daily returns. Use code as appropriate to make these comparisons, and then write out your answer in a markdown cell.

```
# means
mean_worst = monthly["worst_return_pct"].mean()
mean_best = monthly["best_return_pct"].mean()
mean_overall = df["market_return_pct"].mean()

print(mean_worst)
print(mean_best)
print(mean_overall)

-1.7778684429641964
1.8074604496253124
0.04300540006084576
```

Answer

The best and worst case KDEs appear to be mostly symmetrical around $x=0$, suggesting that best and worst daily returns happen at similar frequencies and magnitudes. The best returns KDE has a slightly higher peak, meaning the higher returns occurred more frequently. The KDE of the overall daily US returns in part 1 support the symmetry as the majority of observations are centered at 0 and shows that positive and negative returns occur in same frequencies around 0.

Numerical analysis also supports the symmetry and similarity to the overall returns. The means of the worst and best returns are -1.778 and 1.807 respectively. their magnitudes are very close, and the means themselves sum to a value very close to 0. The mean of the overall returns is also very close to 0. All of this supports the conclusion that gains and losses from the ebb and flow of the market are generally equal in magnitude and frequency.

3. Losses and gains are on opposite sides of zero, which makes them hard to compare directly. Put them on the same scale by looking at the *magnitude* of the worst loss (`-worst_return_pct`) against the magnitude of the best gain (`best_return_pct`). Plot both the KDE and ECDF.

```
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

sns.kdeplot(
    x=monthly["worst_loss_pct"],
    label="Worst loss magnitude",
    cut=0,
    ax=axes[0],
)
sns.kdeplot(
    x=monthly["best_return_pct"],
    label="Best gain",
    cut=0,
    ax=axes[0],
)
axes[0].set(
    title="KDE: magnitudes of monthly extremes",
    xlabel="Return magnitude (%)",
    ylabel="Estimated density",
)
axes[0].legend()

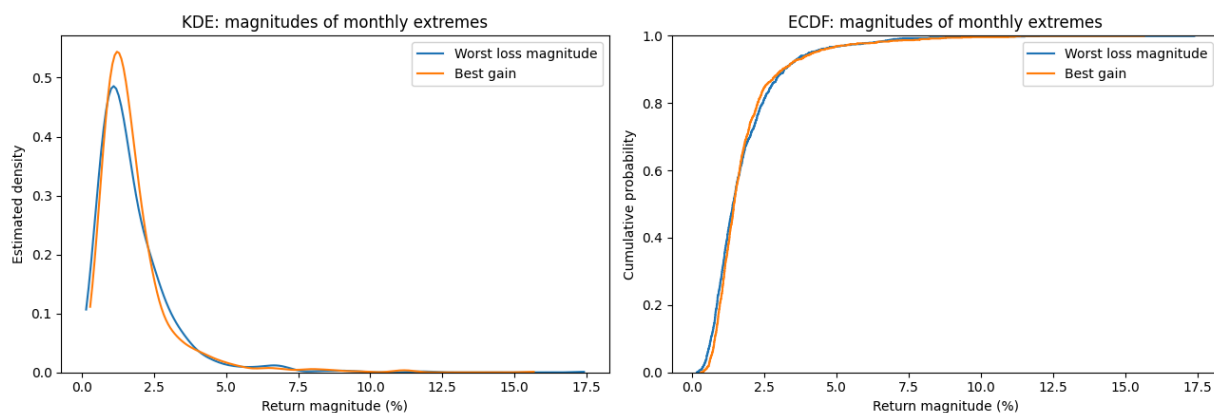
sns.ecdfplot(
    x=monthly["worst_loss_pct"],
    label="Worst loss magnitude",
```

```

    ax=axes[1],
)
sns.ecdfplot(
    x=monthly["best_return_pct"],
    label="Best gain",
    ax=axes[1],
)
axes[1].set(
    title="ECDF: magnitudes of monthly extremes",
    xlabel="Return magnitude (%)",
    ylabel="Cumulative probability",
)
axes[1].legend()

plt.tight_layout()
plt.show()

```



Are large losses more extreme than large gains, or about the same:

Answer

The large overlap of the KDEs and ECDFs suggest that large losses and gains are about the same in return magnitude.

4. Compute the median, 75th, 90th, 95th, and 99th percentile of both magnitudes from question 3. In what fraction of months does the worst loss exceed the best gain in magnitude?

```

tail_summary = pd.DataFrame(
    {
        "Worst loss magnitude": monthly["worst_loss_pct"],
        "Best gain": monthly["best_return_pct"],
    }
).quantile([0.50, 0.75, 0.90, 0.95, 0.99]).T

tail_summary.columns = ["median", "75%", "90%", "95%", "99%"]

print(
    "Fraction of months in which the worst loss exceeded the best gain: "
    f"{(monthly['worst_loss_pct'] > monthly['best_return_pct']).mean():.1%}"
)

```

```

tail_summary.round(2)
Fraction of months in which the worst loss exceeded the best gain: 44.8%

```

	median	75%	90%	95%	99%
Worst loss magnitude	1.41	2.24	2.48	4.06	6.07